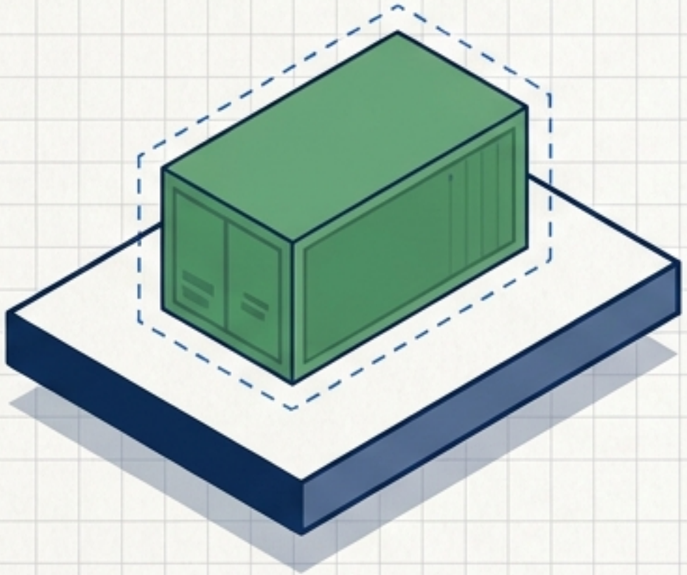
Container Orchestration with Kubernetes

Moving from manual container management to declarative, self-healing systems

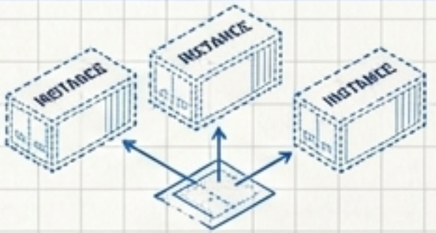
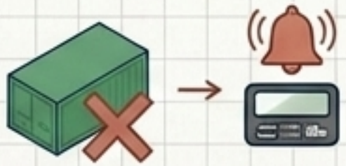
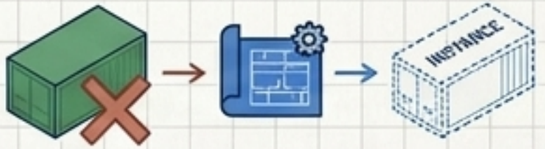
Running one container is easy.
Running hundreds requires an orchestrator.



Manual execution works for isolated instances. You start, stop, and inspect logs manually.

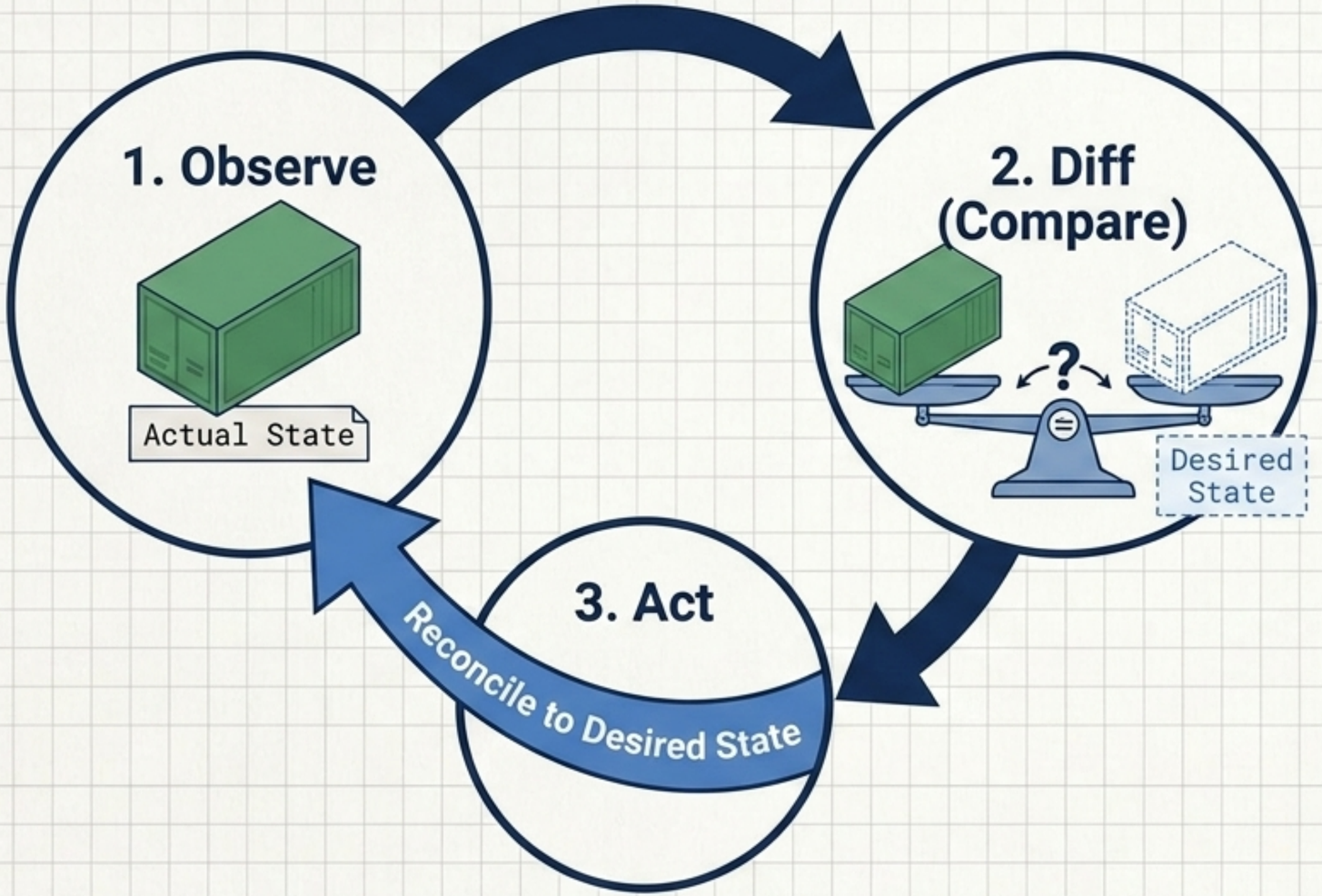
At scale across distributed servers, manual management fails. You need a system to automate placement, restart failed containers, distribute traffic, and execute zero-downtime updates.

The Paradigm Shift: Imperative vs. Declarative

	Imperative	Declarative
Mindset	Start this container right now. 	Maintain three instances of this application. 
Management Style	Manual, brittle, requires constant human monitoring. 	Automated, resilient, driven by desired-state configurations. 
Failure Handling	Container dies → Administrator gets paged. 	Container dies → System automatically replaces it to match the blueprint. 

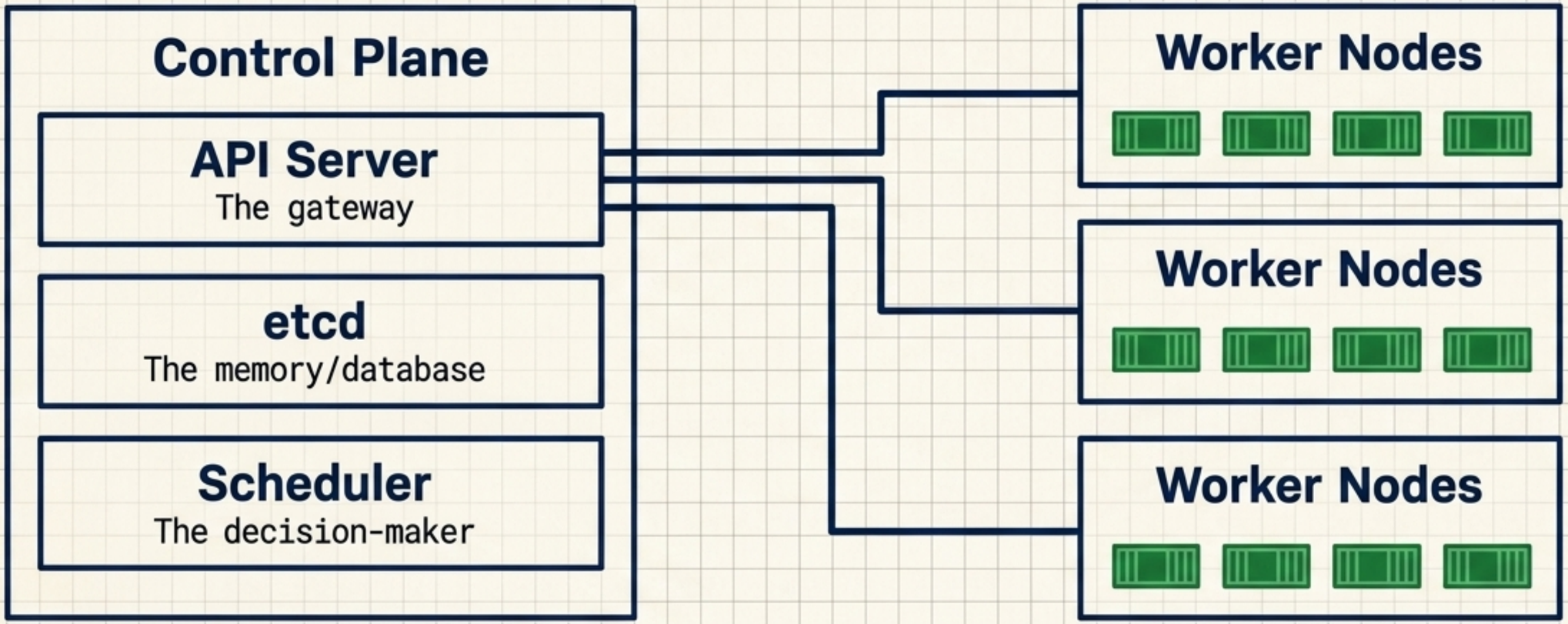
Kubernetes does not just execute commands; it acts as a control system enforcing your declarative blueprint.

The Core Engine: The Reconciliation Loop



Kubernetes does not just execute commands; it acts as a continuous control system. The platform constantly observes the actual cluster state, compares it to the desired state in its database, and takes immediate action to reconcile the differences.

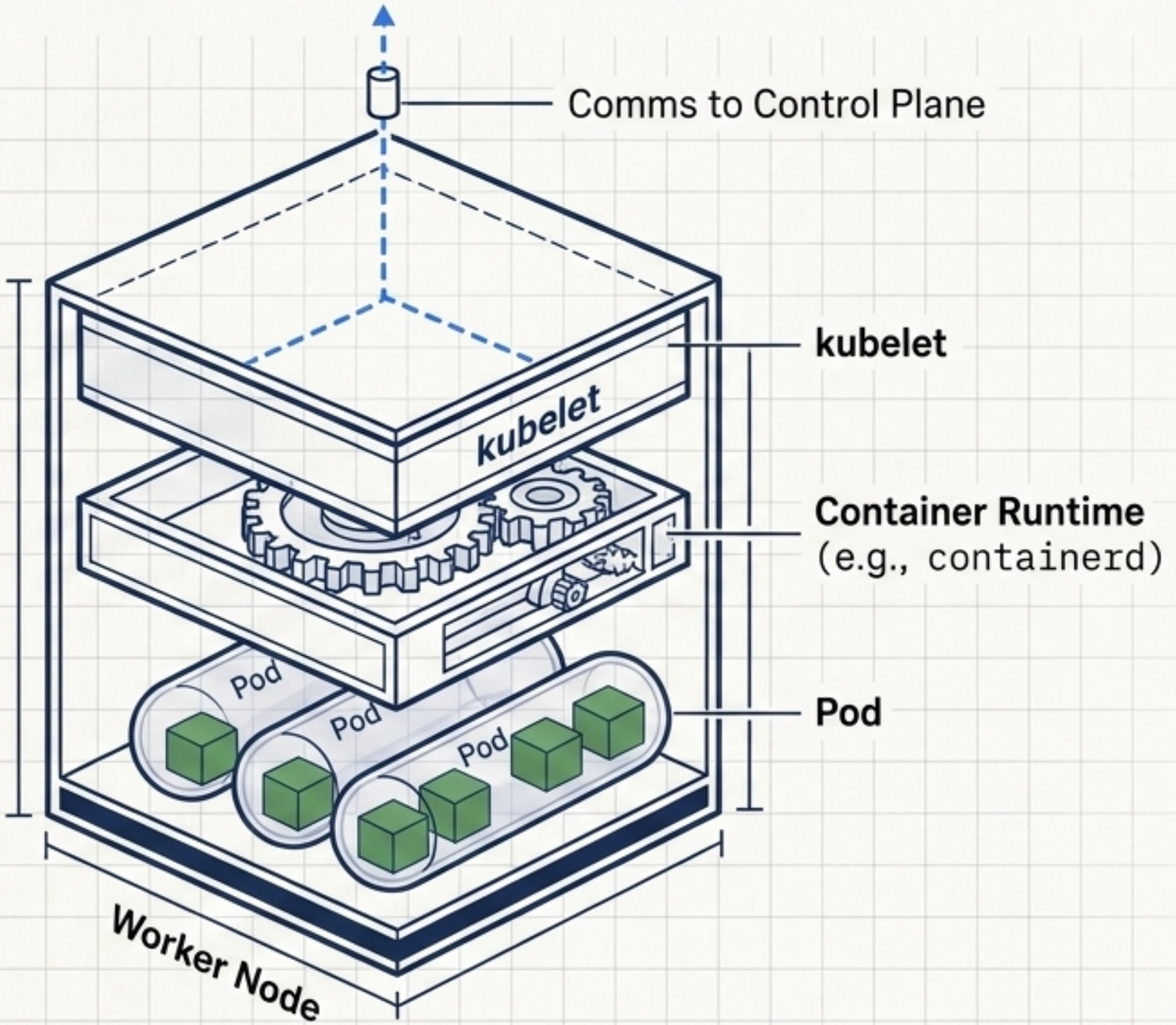
Macro Architecture: Brain and Muscle



The central brain. Makes cluster-wide decisions, validates API requests, and stores the absolute truth of the desired state.

The execution layer. Virtual or physical machines that actually run the application workloads.

Anatomy of a Worker Node

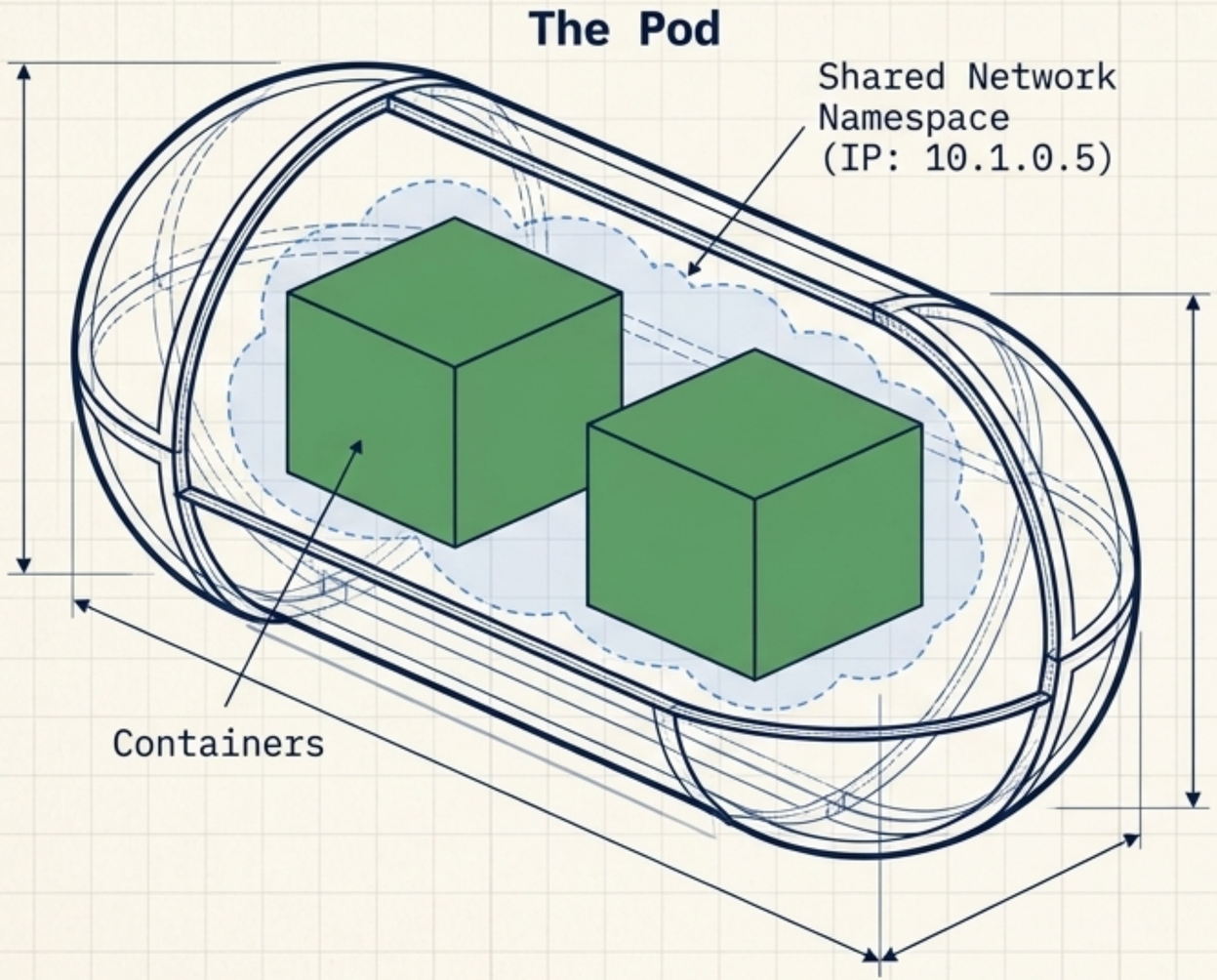


Kubernetes orchestrates; the runtime executes.

Kubelet: The main Kubernetes agent on the node. It takes blueprint orders from the Control Plane and ensures local workloads match the spec.

Container Runtime: The `CRI-compatible` software that actually pulls images and starts container processes.

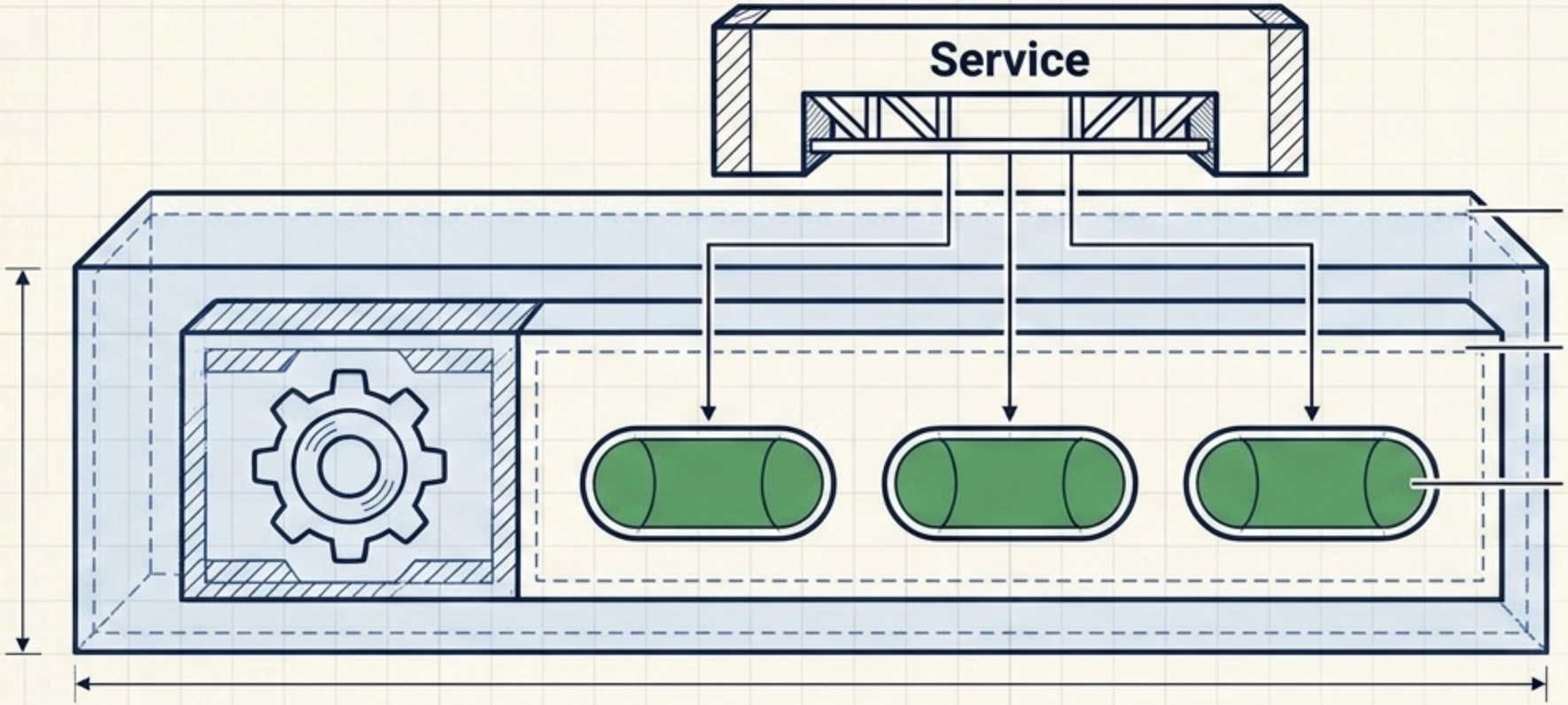
The Fundamental Unit: The Pod



Kubernetes does not schedule containers; it schedules Pods.

- **The Smallest Unit:** A Pod contains one or more tightly coupled containers.
- **Shared Context:** Containers in a Pod share the same network namespace, IP address, and storage volumes.
- **Ephemeral:** Pods are born, live, and die. They are never repaired—only replaced.

The Four Core Objects Synthesized



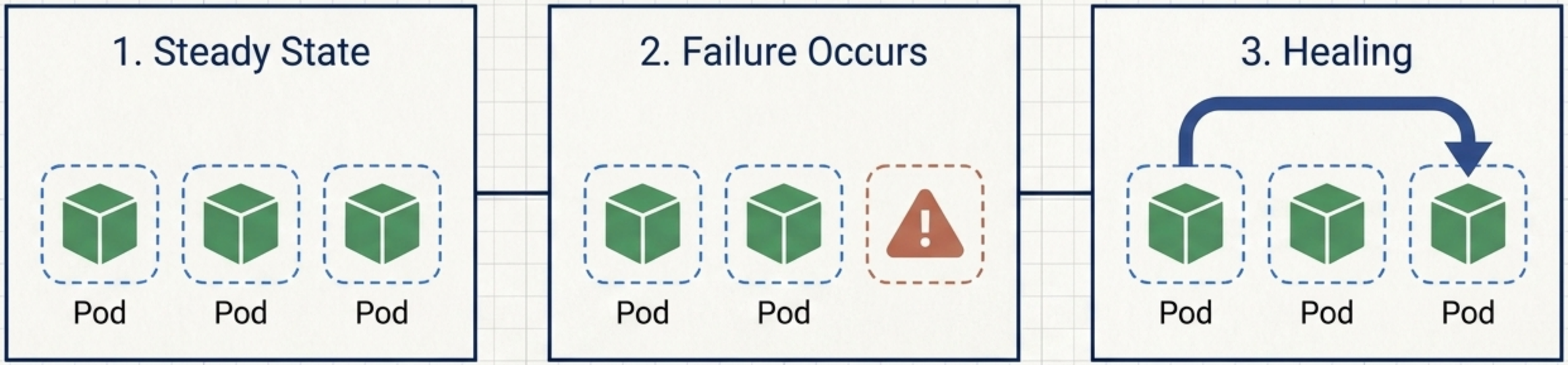
Deployment
(Defines version & strategy)

ReplicaSet
(Maintains exact replica count)

Pods
(Actual running workloads)

The essential vocabulary of application management. A **Deployment** manages the ReplicaSet, which spins up the ephemeral Pods, which are accessed securely via a permanent Service.

Self-Healing & Scaling in Action



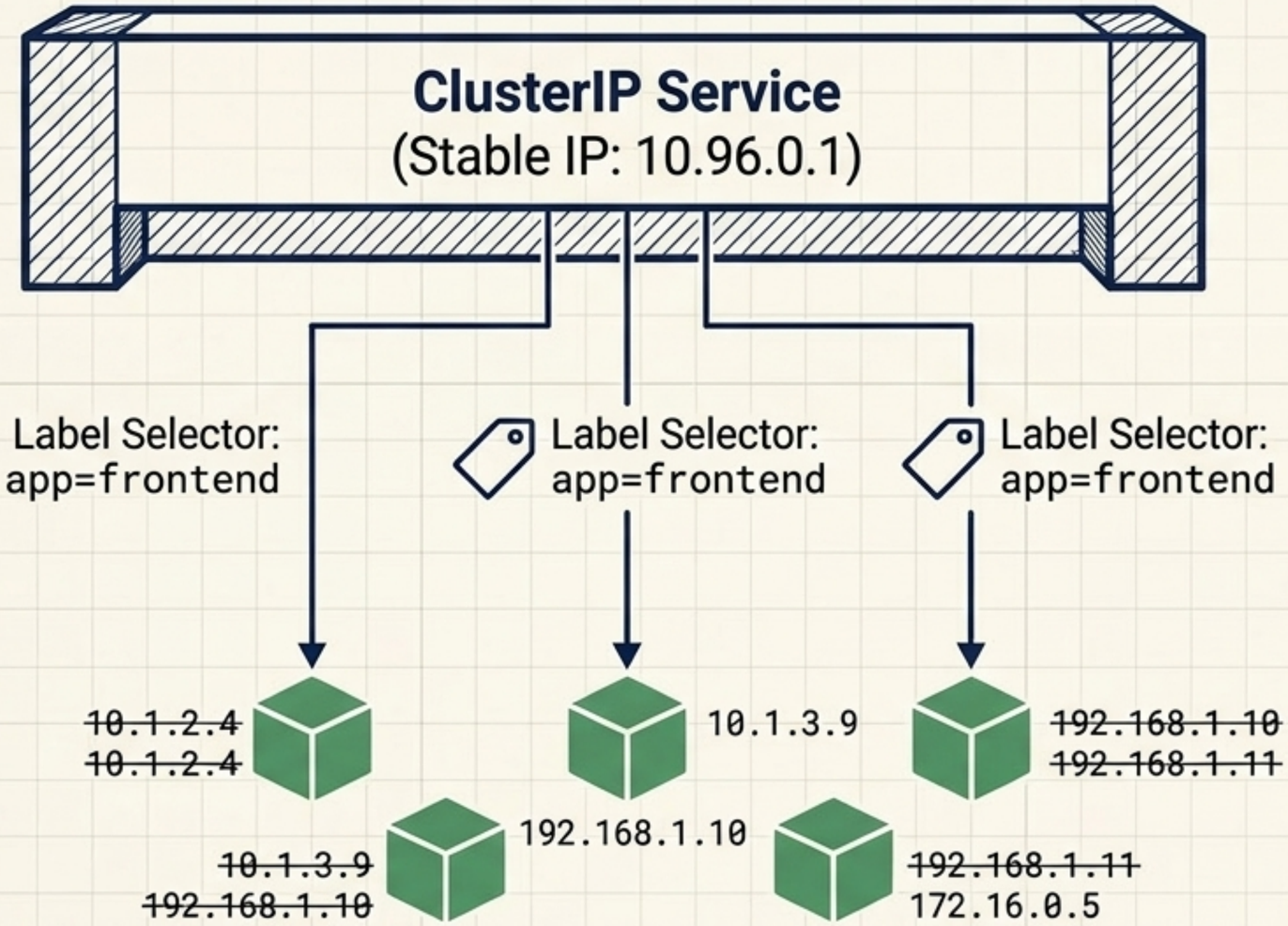
ReplicaSet desires 3. Actual is 3.

A node fails. Actual Pods drop to 2. The ReplicaSet detects the deviation.

A new Pod is instantly scheduled on a healthy node to restore the balance. 3=3.

Kubernetes automatically replaces failed workload instances according to the configured desired state.

Services & Networking: Order from Chaos



Because Pods are ephemeral, their IP addresses constantly change.

A Service provides a stable, permanent network abstraction.

Labels identify resources; **Selectors** connect them.

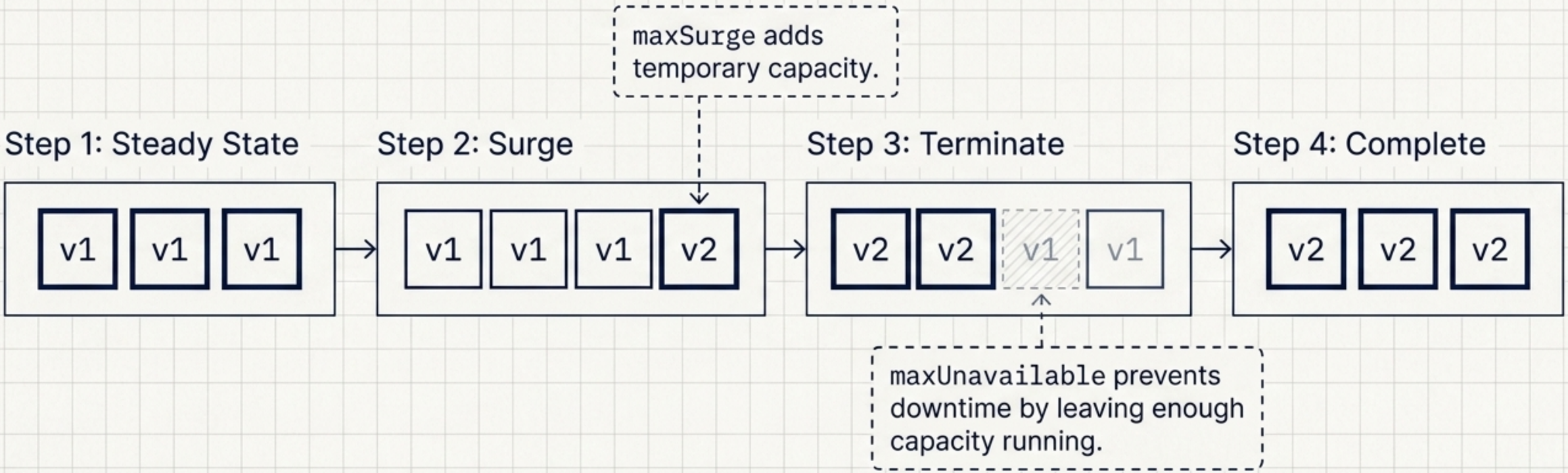
The Service dynamically routes traffic to any active Pod that matches its Label Selector.

Application Health: Readiness vs. Liveness

Readiness Probe	Liveness Probe
	
Should this Pod receive user traffic right now?	Is this container hopelessly deadlocked or stuck?
Stops routing traffic to the Pod. Prevents sending users to an app that is still booting up.	Restarts the container process. Triggers a reboot if the app crashes but the process stays alive.

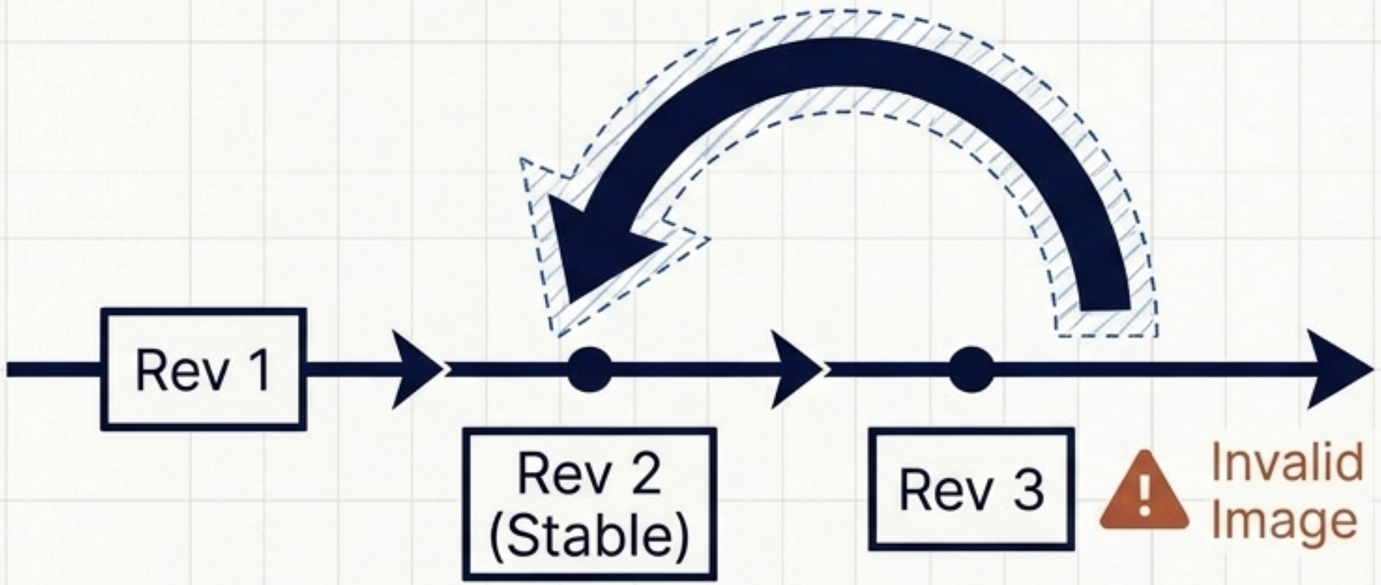
Correctly configuring both probes ensures users never hit an unready application and deadlocked containers are automatically healed.

Evolution: The Rolling Update



Gradual replacement of Pods ensures zero downtime. The Deployment controller creates a new ReplicaSet and carefully scales it up while scaling down the old one, governed by strict capacity constraints.

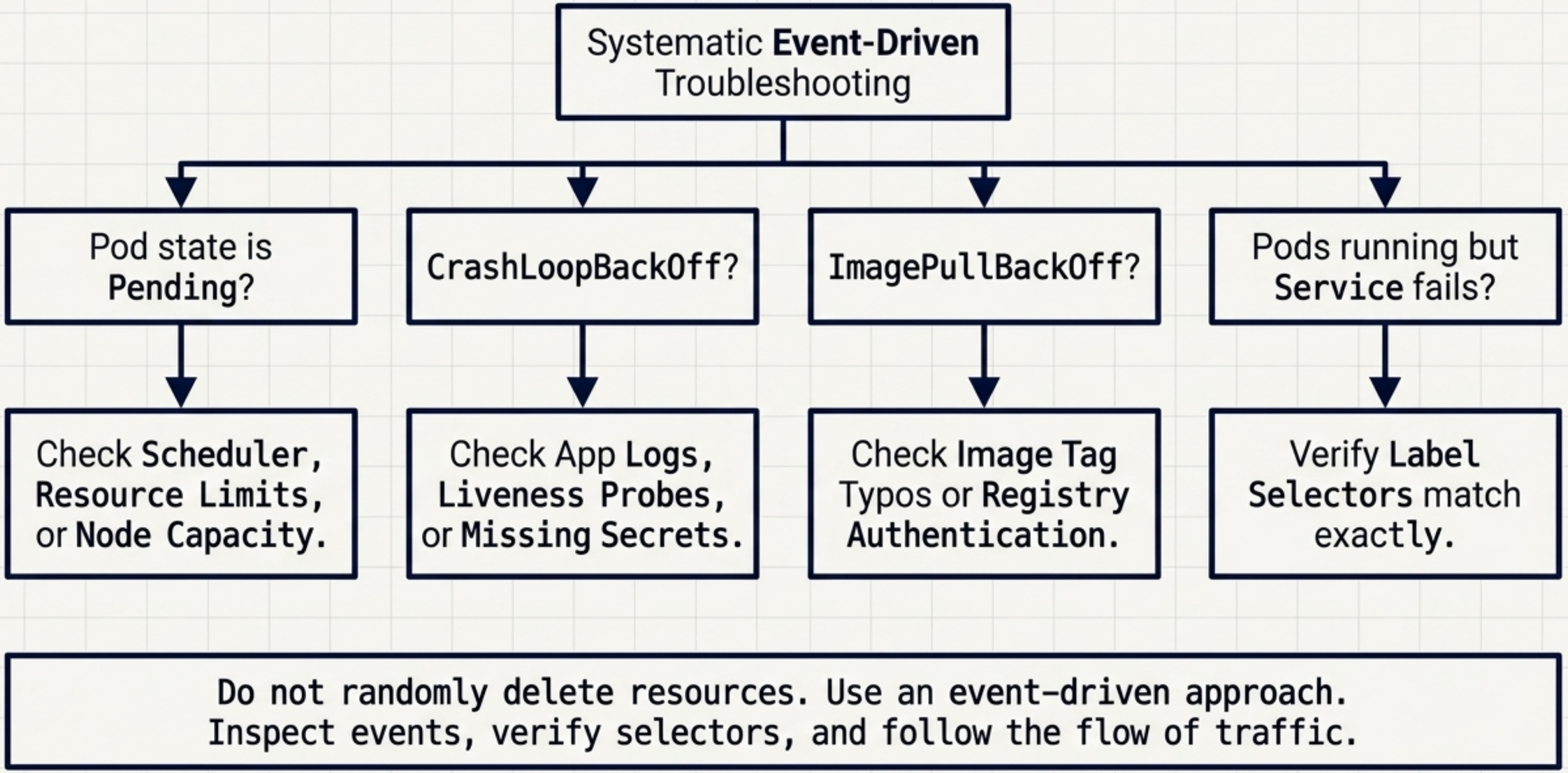
Time Travel: Rollbacks & Revisions



Crucial Note: Rollbacks restore workload configuration, not persistent data. If an update altered a database schema, simply rolling back the application Pods may cause fatal incompatibilities.

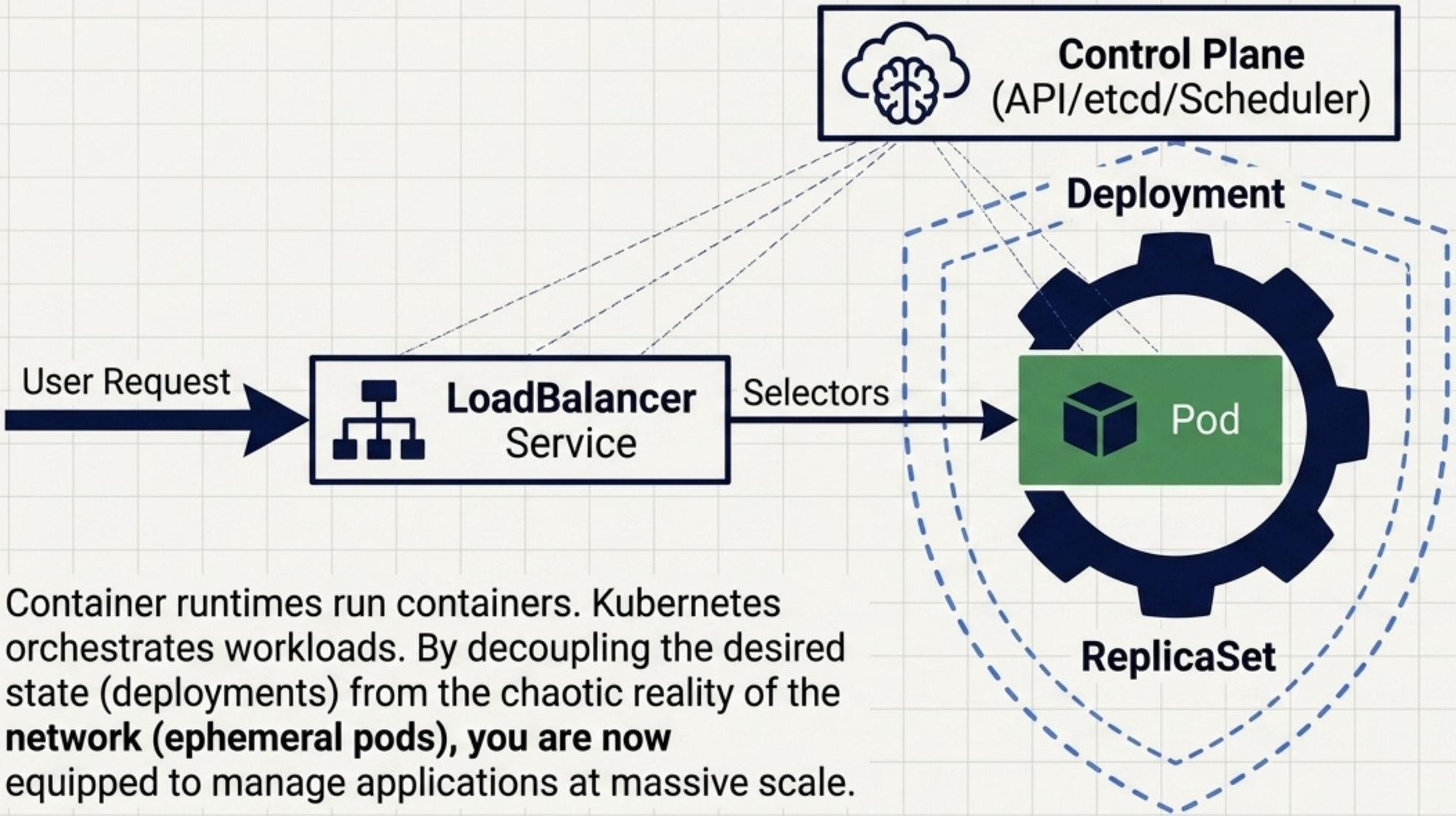
Deployments maintain a strict revision history. If an update fails (e.g., `CrashLoopBackOff`), Kubernetes can instantly roll back to a known-good previous desired state.

Troubleshooting Diagnostic Tree



Do not randomly delete resources. Use an event-driven approach. Inspect events, verify selectors, and follow the flow of traffic.

Final Synthesis: The Automated Blueprint



Container runtimes run containers. Kubernetes orchestrates workloads. By decoupling the desired state (deployments) from the chaotic reality of the **network (ephemeral pods), you are now** equipped to manage applications at massive scale.